

Proyecto #2 2026: Pintar la valla – Cobertura de subconjuntos de costo mínimo

Javier Eduardo España Pacheco (23361)
Angel Esteban Esquit Hernández (23221)
Roberto José Barreda Siekavizza (23354)

Mayo 2026

Resumen

Se presenta la formulación por subconjuntos del problema de pintar una valla discretizada en n secciones. Se describen el algoritmo exacto (programación dinámica sobre subconjuntos) y una heurística greedy, se analizan sus complejidades y se muestran resultados empíricos que comparan tiempos y calidad de solución. Además se incluye un contraejemplo que ilustra la suboptimalidad de la heurística.

1. Introducción

En este informe se estudia la variante por subconjuntos del problema de cobertura de costo mínimo aplicado al problema de pintar una valla. Se exponen motivaciones, contribuciones y la estructura del documento.

2. Formulación del problema

Instancia: Un entero n (número de secciones) y un conjunto de m pintores, cada uno definido por una máscara de secciones $mask_i$ y un costo $c_i > 0$.

Objetivo: Seleccionar un subconjunto de pintores cuya unión cubra todas las n secciones con costo total mínimo.

3. Análisis del problema

3.1. Subestructura óptima

Sea $dp[mask]$ el costo mínimo para cubrir exactamente el subconjunto de secciones indicado por $mask$. Una solución óptima para un estado $mask$ se obtiene añadiendo un pintor i a una solución óptima de otro estado anterior $prev$, con $mask = prev \cup mask_i$.

Por tanto, el problema presenta subestructura óptima: si existiera una mejor solución para el estado previo, entonces también existiría una mejor solución para el estado actual.

3.2. Relación de recurrencia

Se modela la valla como un conjunto de n secciones discretas. Cada pintor i cubre un subconjunto arbitrario de esas secciones, representado mediante una máscara de bits, y tiene un costo positivo

c_i . El objetivo es seleccionar un subconjunto de pintores cuya unión cubra todas las secciones y cuyo costo total sea mínimo.

Esta formulación es equivalente a una instancia de cobertura sobre subconjuntos y conduce de manera natural a un algoritmo exacto exponencial en n . En los experimentos del proyecto se trabaja directamente con máscaras de bits, por lo que la programación dinámica recorre el espacio de subconjuntos cubiertos.

La recurrencia exacta es:

$$dp[mask \cup mask_i] = \min(dp[mask \cup mask_i], dp[mask] + c_i),$$

donde $mask$ recorre todos los subconjuntos alcanzables y $mask_i$ es la máscara del pintor i . El estado inicial es $dp[0] = 0$ y todos los demás estados comienzan en ∞ .

4. Algoritmos de solución

4.1. Programación dinámica exacta

Algoritmo 1 DP exacta para cobertura de subconjuntos

Requerir: n secciones y pintores con máscaras $(mask_i, c_i)$

Garantizar: Costo mínimo y conjunto de pintores elegidos

- 1: Inicializar $dp[0..2^n - 1] \leftarrow \infty$
 - 2: $dp[0] \leftarrow 0$
 - 3: **Para** cada estado $mask$ alcanzable **hacer**
 - 4: **Para** cada pintor i **hacer**
 - 5: $nuevo \leftarrow mask \vee mask_i$
 - 6: $dp[nuevo] \leftarrow \min(dp[nuevo], dp[mask] + c_i)$
 - 7: **fin Para**
 - 8: **fin Para**
 - 9: Reconstruir la solución siguiendo los punteros guardados
-

Su tiempo de ejecución es $O(2^n \cdot m)$, donde n es la cantidad de secciones y m el número de pintores. Por depender exponencialmente de n , es una solución verdaderamente superpolinomial.

4.2. Greedy

Algoritmo 2 Heurística greedy por cobertura nueva/costo

Requerir: n secciones y pintores con máscaras $(mask_i, c_i)$

Garantizar: Cobertura heurística

- 1: $covered \leftarrow 0$
 - 2: **Mientras** $covered \neq 2^n - 1$ **hacer**
 - 3: Seleccionar el pintor que maximice $\frac{|mask_i \setminus covered|}{c_i}$
 - 4: Agregarlo a la solución y actualizar $covered \leftarrow covered \vee mask_i$
 - 5: **fin Mientras**
-

En la implementación usada para los experimentos, la heurística recorre todos los pintores en cada elección, por lo que su complejidad es $O(m^2)$ en número de comparaciones, con costo adicional

lineal en el tamaño de las máscaras; en notación gruesa, puede expresarse como $O(m^2n)$. Aun así, resulta mucho más rápida en la práctica que la DP exacta.

5. Discusión sobre la heurística greedy

El problema no presenta la **greedy choice property** (propiedad de elección golosa). Esta propiedad no se cumple en general para el problema ponderado. El criterio local de mayor cobertura nueva por costo puede escoger un pintor barato y aparentemente conveniente que obliga a pagar más en los pasos siguientes, mientras que una opción ligeramente menos atractiva a corto plazo puede conducir a una solución globalmente más barata.

Los resultados empíricos se presentan en la siguiente subsección, donde se muestran la comparación de tiempos, el ajuste teórico seleccionado y el contraejemplo usado para ilustrar el comportamiento de la heurística.

5.1. Ajuste teórico

Para cada algoritmo se compararon formas candidatas acordes con su complejidad esperada. En DP se probaron familias exponenciales y de potencia; en Greedy se probaron formas cuadráticas y $n \log n$. El ajuste elegido se seleccionó minimizando el Criterio de Información Bayesiano ($BIC = n \ln(MSE) + k \ln n$, con k parámetros y n mediciones), que penaliza parámetros adicionales más fuertemente que el R^2 ajustado.

Los nombres exactos del ajuste seleccionado, su ecuación estimada y sus métricas aparecen en la tabla empotrada más abajo, junto con el resto de los resultados empíricos.

6. Conclusiones

La programación dinámica sobre subconjuntos ofrece la solución óptima, pero su costo crece exponencialmente con el número de secciones ($\Theta(2^n)$). La heurística greedy es significativamente más rápida en la práctica y suele producir soluciones cercanas a la óptima, aunque no garantiza optimalidad. En proyectos con restricciones fuertes de tiempo, greedy es apropiada; cuando se requiere certidumbre sobre optimalidad, la DP exacta es necesaria.

Implementación y artefactos del repositorio

En el repositorio se documenta la implementación de esta variante exacta (`SolveExactDp_Subsets`) y de su heurística asociada (`SolveGreedyHeuristic_Subsets`), además del generador de instancias factibles (`GenerateFeasibleInstance_Subsets`), pruebas unitarias y un benchmark que vuelca resultados en `results/`.

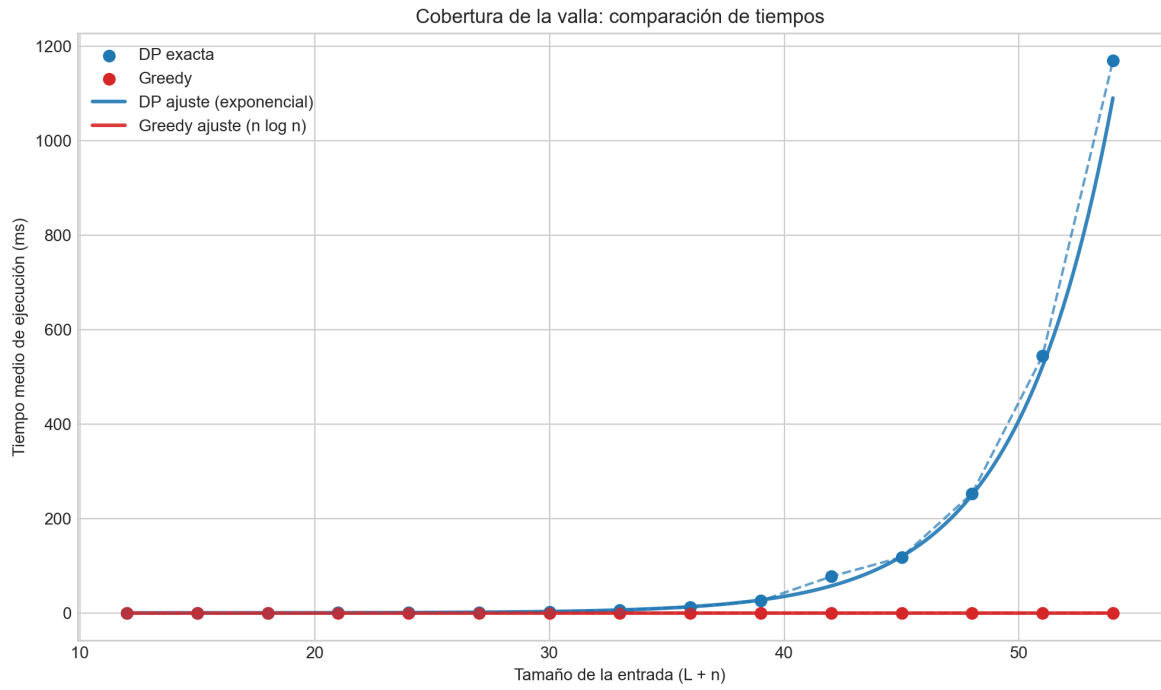


Figura 1: Tiempos de ejecución para la variante por subconjuntos (exponencial).

6.0.1. Resultados empíricos para subconjuntos

oprule Sections	m	n	DP med. (ms)	Greedy med. (ms)	Razón media
4	8	12	0.0570	0.0318	1.0000
5	10	15	0.0745	0.0293	1.0000
6	12	18	0.1582	0.0408	1.0000
7	14	21	0.3087	0.0362	1.0000
8	16	24	0.5056	0.0314	1.0000
9	18	27	1.1893	0.0399	1.2500
10	20	30	2.6449	0.0440	1.1667
11	22	33	5.8743	0.0559	1.0000
12	24	36	12.5597	0.0420	1.3333
13	26	39	25.6113	0.0345	1.0000
14	28	42	77.1101	0.0445	1.0000
15	30	45	117.9755	0.0375	1.0000
16	32	48	252.9807	0.0449	1.0000
17	34	51	543.7876	0.0492	1.1667
18	36	54	1169.1992	0.0723	1.0000

Contraejemplo (variante por subconjuntos). Instancia encontrada por búsqueda aleatoria (máscara y costo):

oprule	Nombre	Mascara	Costo
	b0	0b1	2
	b1	0b10	3
	b2	0b100	3
	b3	0b1000	2
	b4	0b10000	3
	b5	0b100000	2
	b6	0b1000000	3
	b7	0b10000000	2
	b8	0b100000000	3
	b9	0b1000000000	3
	b10	0b10000000000	3
	r11	0b1101100	6
	r12	0b1000000110	2
	r13	0b1101001	6
	r14	0b10100111101	10
	r15	0b110101	3

La solución óptima exacta cuesta 17 y la heurística greedy cuesta 18.

7. Enlaces de entrega

Repositorio de código: <https://github.com/usuario/repo> (Reemplazar con enlace real)

Videos individuales (YouTube): El video de cada integrante debe cubrir: problema de cobertura por subconjuntos, subproblemas, criterio greedy, diferencias entre greedy y DP, y la calidad observada de la heurística.

- Javier España: <https://youtube.com/...> (Reemplazar con enlace real)
- Angel Esquit: <https://youtube.com/...> (Reemplazar con enlace real)
- Roberto Barreda: <https://youtube.com/...> (Reemplazar con enlace real)